Piece of History

Master Build Spec — Version 2 (Snowy Blue UI + Cumulative Real-Money Bidding)

What Changed in v2

- **Visual design:** The product now follows the Snowy Blue UI direction (icy blue + white, navy text/buttons, snowflakes, mountains, playful illustrations, funny micro-copy) instead of the earlier futuristic/electric-blue direction. All core functionality still comes from the original build prompt.
- **Bidding logic:** Bidding is now cumulative and always real money. Every payment a participant makes is charged immediately and is non-refundable. A participant's standing for the day is the running total of everything they have paid that day, not just their latest single payment. Whoever has the highest running total is #1.
- **Traffic to participants:** The daily competition list keeps a prominent "Visit Website" action on every participant entry (this was already part of the original spec and is reconfirmed here as a hard requirement, not an optional nice-to-have).

Core Tagline

"Every day disappears. Some days become history."

1. Visual Design Direction — Snowy Blue

Use this direction for every screen. It replaces any earlier "deep navy / electric blue / glass card" direction mentioned in older notes.

Palette & mood

- Backgrounds: mostly white and very pale icy blue.
- Text & primary buttons: deep navy blue for strong contrast.
- Accents: soft gradients, subtle snowflakes, snow-covered mountains, small friendly illustrations (e.g. a penguin, a snowman) used sparingly so the UI stays clean and fast, never visually heavy.
- Tone: playful and a little funny, not corporate. Short, witty micro-copy throughout.

Example micro-copy to reuse or riff on

- “Some days are just too good to keep to yourself…”
- “Bid on a date. Make it yours. Make history.”
- “Higher the bid. Stronger the story.”
- “Not just dates. It’s bragging rights for life.”
- “Got a crazy plan? Make it a date. Make it history!”
- “Life is short. Bid on the moments that matter.”

Reference screens (from the UI guide)

- **Landing / Home** — hero headline, short pitch, “Start Your Story” CTA, a 4-step strip (Create your story → Choose a date → Let people bid → Win & make it history), snowy illustration with a signpost/penguin.
- **Sign in / Welcome back** — simple email + password form, Google/GitHub continue buttons, snowman illustration.
- **Explore** — greeting header, search bar, filter chips (All / Winning / Ongoing / Upcoming), a grid of story cards each showing status, title, price, and a CTA.
- **Competition detail** — countdown timer, current top bid, “Place a Bid” CTA, and a running bid-history list of participants with their amounts and timestamps.
- **Create Your Story** — a short form: title, description, suggested date.

2. Core Product Rules

There is one active day at a time. Each participant submits a short story and an optional/required website URL, then competes to be #1 for that day using the cumulative bidding model described in Section 3. At exactly 23:59:59 in the site's configured timezone, the day freezes permanently and whoever is #1 at that instant wins the day's public Piece of History. The winning outcome (and every payment collected that day) goes to the site owner — no platform fee, no revenue share, no subscriptions.

3. NEW — Cumulative Real-Money Bidding Logic

This replaces the old “only the winner ever pays” rule. Every bid is a real, immediate, non-refundable payment. A participant's rank is driven by the sum of everything they have paid that day, not by their most recent payment alone.

Worked example

Time	Actor	Payment	New running total	Who is #1
10:00	A	$5	A = $5	A
11:30	B	$10	B = $10	B (10 > 5)
14:00	A	$6	A = $5 + $6 = $11	A (11 > 10)

Rules to implement

- **Ledger, not a single field:** every successful payment is written as its own immutable row (user, day, amount, timestamp, payment reference). Never overwrite a previous bid — the running total is a SUM over that ledger.
- **Standing = SUM(amount) per user per active day.** The leader is the participant with the highest standing. Recompute this transactionally every time a new payment is captured.

- **Every payment is charged immediately, win or lose.** There is no “harge only the final winner” flow anymore — remove that assumption everywhere it appears in the original build prompt (including the payment section below). Make this unmistakably clear to users at the moment they pay: this money is spent whether or not they end up #1.
- **Minimum increment:** require a configurable minimum top-up (e.g. at least enough to strictly exceed the current leader's standing, or a flat minimum such as $1) so a challenger cannot dribble in $0.01 payments to spam the leaderboard.
- **Tie-break rule:** if two participants reach the exact same standing, the one who reached that total first (earliest timestamp) keeps or takes #1. Enforce this with a database-level, transactional check so two simultaneous payments cannot both believe they are #1.
- **Concurrency:** wrap “capture payment → write ledger row → recompute leader” in a single atomic transaction with row-level locking on the day record, so concurrent payments cannot corrupt the leaderboard.
- **Personal history still shows every individual payment** a user made that day, plus their running total and final rank — not just one bid amount.

> Compliance note (not legal advice): charging real, non-refundable money for every competitive bid — where only one side gets the tangible outcome — can resemble regulated gambling/lottery mechanics in some jurisdictions, including India. Have this specific mechanic reviewed by a lawyer familiar with local gaming/auction law before accepting real payments, and confirm your chosen payment provider's terms permit it.

4. Personal History

Every authenticated user has a private “My History” page listing every day they participated in: their story, every individual payment they made that day, their final standing, and whether they won or lost. No followers, likes, comments, badges, points, or messaging.

5. Competition List + Traffic to Participants

The active day shows a clear list: #1 leader prominently at the top, then the rest of the participants below. Every entry shows display name, story, current standing, current rank, and — if they supplied one — a prominent “Visit Website” button that opens their site in a new tab with rel="noopener noreferrer". This is a hard requirement: the product should function as a genuine referral source for participants, not just a bidding page. The winner's archived page keeps this link forever.

6. Public Archive / Frozen History

Once a day closes, its page becomes an immutable public record: date, winner, winning story, final winning standing, participant count, and the winner's website link. Older days are browsable through a simple archive/calendar list. A closed day can never be edited through normal user actions.

7. Site Identity & Curiosity

Keep the creator anonymous — no real name, face, or personal profile. A small footer line such as “Developed by …” can hint at this without revealing anything. Show only real, live statistics (days launched, users, pieces of history) — never fabricated numbers.

8. Payment System (Updated for Cumulative Bidding)

- Every payment (not just the winning one) must be authorized and captured before it is written to the bidding ledger. Never trust a client-side “payment successful” message — use the payment provider's server-side webhook as the source of truth.
- For an India-based owner, evaluate Razorpay first: it supports cards, international payments, and settles to a registered bank account; confirm exact fees/eligibility during onboarding.
- Build the payment layer behind a small provider-abstraction interface so a second provider could be added later, but only implement one provider in the MVP.
- The owner gets a minimal private admin view of successful payments, running totals, and payment status per day — not a large dashboard.

9. Scale (~100,000 users)

- Production-grade managed PostgreSQL, proper indexes (especially on user_id + day_id for the ledger), connection pooling, pagination, and stateless app instances.
- The “capture payment → update leaderboard” and “close the day” operations must be transactional and idempotent so retries can never create duplicate winners or corrupt totals.
- Server time only for the countdown and the close deadline — never trust the client's clock.
- Rate limiting, bot protection, input validation, HTTPS, secure cookies, CSRF protection, secret management, structured logging, error monitoring, and backups.

10. Recommended Tech Stack (Replit-first)

- **App framework:** Next.js (App Router) + TypeScript + Tailwind CSS — runs natively on Replit, one codebase for UI + API routes.
- **Database:** Replit's built-in PostgreSQL database (Neon-backed) + Prisma ORM for schema, migrations, and the transactional ledger queries.
- **Auth:** NextAuth.js (email/password plus Google/GitHub OAuth) — keeps identity simple while supporting anonymous public display names.
- **Payments:** Razorpay via a small server-side abstraction layer; webhooks verified server-side as the source of truth for every payment.
- **Live leaderboard updates:** start with short-interval polling (e.g. every 3–5 seconds) for the MVP; only move to websockets/Pusher later if real load requires it — keep the MVP simple.
- **Scheduled day-close job:** a Replit Scheduled Deployment (cron) that runs the idempotent “close day” transaction at 23:59:59 in the configured timezone.
- **Testing:** Vitest/Jest for unit + transaction tests, Playwright for the critical end-to-end flows.

- **Deployment:** Replit Autoscale Deployment for the web app; environment variables/Secrets for all credentials.

11. Updated Data Model

- **User** — id, display name, email, auth provider, created_at.
- **DailyPiece** — id, date, closing_timestamp (server time), status (active/closed), winner_user_id, winning_standing, closed_at.
- **Story** — id, user_id, daily_piece_id, display name, story text, website_url, created/updated timestamps, status.
- **Bid (ledger row)** — id, user_id, daily_piece_id, amount, created_at, payment_reference, payment_status. Immutable once captured.
- **ParticipantStanding** (computed/materialized) — user_id, daily_piece_id, running_total = SUM(amount) from Bid, last_payment_at (used for tie-breaks).
- **AuditEvent** (optional) — for tracking leader changes, day closes, and admin actions.

12. Do Not Build

No social network, followers, comments, likes, chat, subscriptions, points, NFTs, crypto, AI-generated stories, badges, marketplace, revenue sharing, proxy/max bidding, multi-level memberships, or extra gamification. This stays a tiny, memorable side project with one extraordinary mechanic.

13. Final Instruction to the AI Coding Agent

Build this as a real, production-quality MVP on Replit, in the Snowy Blue visual direction, with the cumulative real-money bidding ledger described above as the core mechanic. When a decision is ambiguous, prefer the smallest implementation that preserves: participants pay real, non-refundable money that accumulates into a running total per day; the highest running total at 23:59:59 wins; every participant's website is discoverable and clickable throughout the competition and forever after in the archive; the site owner keeps every payment collected. Do not invent extra product concepts.